

ACCOUNTING RECEIVABLE SYSTEM



UNIVERSITY OF MINDANAO
College of Engineering
Computer Engineering Program

In Partial Fulfillment of the Requirement for the course
CpE 211/L- Data Structures and Algorithm

Jireh Lie Cabrestante
Elijah Joseph Orias
Patricia Madeth Buna

March 2021

TABLE OF CONTENTS

Software Description.....	3
Flowchart.....	4
File Description.....	5
Source Code.....	6-11
Screen Output.....	12-16
References.....	17

SOFTWARE DESCRIPTION

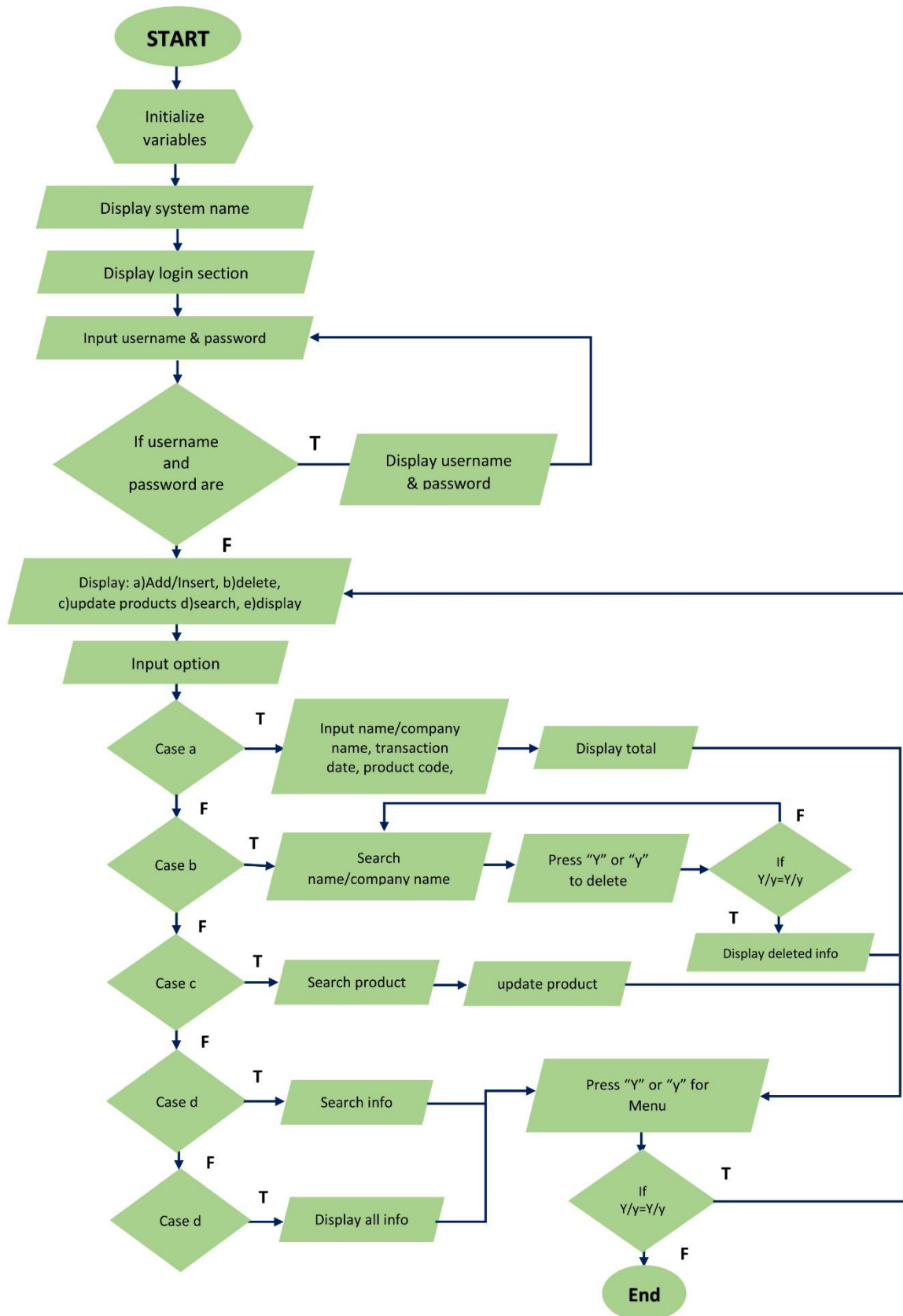
The Accounting Receivable System encompasses the recording of a customer's information and its purchased products. The system contains a menu with five buttons namely – Add, Products, Update, Search and Display. The Add Button enables the user to input its necessary details such as name, address, phone number and email. The customer can view all products available to purchase through the Products Button. It also views all the information of each products with its code, name, quantity, and price. The Update Button allows the user to update the customer's product and payment information. In searching some details, the user can proceed to the Search Button. Then, the Display Button views all stored details the customer had input as well as the payments. All details that were sent for input into the system shall be kept confidentially and secured.

Accounting Receivable System can keep track of details sent, payments received, and any customers that may be past due on their bills based on the indicated date. The customer can also customize its preferences based on their input to the Add Button. Though there can be limitations on the system. There are a few examples like - On the Search Button, you can only search via date. While on the Update Button, you can search via name or code.

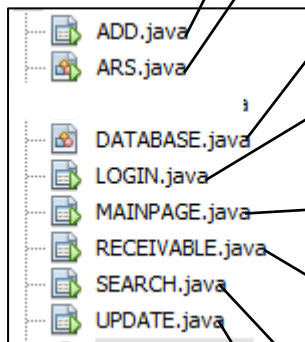
The system is surely created for comfort and with easy access among customers. It is user-friendly, with a clean design yet robust set of features that helps users generate actionable, data-based insights, and includes invoicing capabilities and receipt collection for good measure. Also, it is systematically fast, comprehensive, detailed for financial reports, and automated record keeping.

The platform requirements for the system, Accounting Receivable System are as follows: The OS should be Windows 7 with SP1; [Recommended: Windows 10]. The CPU must be Intel or AMD processor with 64-bit support; [Recommended: 2.8 GHz or faster processor]. The GPU must be nVidia GeForce GTX 1050 or equivalent; [Recommended: nVidia GeForce GTX 1660 or Quadro T1000]. The Disk Storage should have 4 GB of free disk space with a Monitor Resolution of 1280x800; [Recommended: 1920x1080].

FLOWCHART



FILE DESCRIPTION



ADD Frame is where the customers information like name, address, etc. are being inputted.

ARS Class is the main class of the system where the LOGIN Frame can be called.

DATABASE Class is where the connection towards the database is being established.

LOGIN Frame is where the admin enters their id and password to access the system.

MAINPAGE Frame is the menu of the system where ADD, UPDATE, RECEIVABLE, SEARCH buttons are being displayed.

RECEIVABLE Frame is where we can view the monthly and yearly total receivables from the customers.

SEARCH Frame is where we can search and display customers information.

Update Frame is where we can update and delete customers information.

SOURCE CODE

```

1 package ars;
2 import java.sql.Connection;
3 import java.sql.DriverManager;
4 import java.sql.SQLException;
5 import javax.swing.JOptionPane;
6 public class DATABASE {
7     final static String JDBC_DRIVER = "com.mysql.jdbc.Driver";
8     final static String DB_URL = "jdbc:mysql://localhost:3306/ars";
9
10    final static String USER = "root";
11    final static String PASS = "";
12    public static Connection connection(){
13        try{
14            Class.forName(JDBC_DRIVER);
15            Connection conn = DriverManager.getConnection(DB_URL,"root","");
16            System.out.println("Connected");
17            return conn;
18        }catch(ClassNotFoundException | SQLException e){
19            JOptionPane.showMessageDialog(null, e);
20            return null;
21        }
22    }
23
24 }
25
26 }
27

```

DATABASE Code

```

1
2 package ars;
3
4
5 public class ARS {
6
7     public static void main(String[] args) {
8         LOGIN gui = new LOGIN();
9         gui.setTitle("Login");
10        gui.setLocationRelativeTo(null);
11        gui.setVisible(true);
12    }
13
14 }
15

```

ARS Class Code

```

1 package ars;
2 import java.awt.HeadlessException;
3 import java.sql.Connection;
4 import java.sql.ResultSet;
5 import java.sql.SQLException;
6 import java.sql.Statement;
7 import javax.swing.JOptionPane;
8 public class LOGIN extends javax.swing.JFrame {
9     Connection connect = null;
10    Statement state = null;
11    ResultSet result = null;
12    public LOGIN() {
13        super("LOGIN");
14        connect = DATABASE.connection();
15        initComponents();
16    }
17    public void Login(){
18        try {
19            state = connect.createStatement();
20            String ID = id.getText();
21            String password = pass.getText();
22
23            String sql = "SELECT * FROM login WHERE adminid = '"+ID+"'&& adminpass= '"+password+"' ";
24            result = state.executeQuery(sql);
25            if(result.next()){
26                this.setVisible(false);
27                String getter = ID;
28                new MAINPAGE(getter).setVisible(true);
29            }
30            else{
31                JOptionPane.showMessageDialog(null,"ID or PASSWORD is invalid.");
32            }
33        }
34        catch (HeadlessException | SQLException e){
35        }
36    }
37 }

```

LOGIN Frame Code

```

1 package ars;
2 import java.sql.Connection;
3 import java.sql.ResultSet;
4 import java.sql.SQLException;
5 import java.sql.Statement;
6 import javax.swing.JOptionPane;
7 public class MAINPAGE extends javax.swing.JFrame {
8     Connection connect = null;
9     Statement state = null;
10    ResultSet result = null;
11    String getter;
12    public MAINPAGE( String ID) {
13        initComponents();
14        getter = ID;
15        try{
16            connect = DATABASE.connection();
17            state = connect.createStatement();
18            String sql = "SELECT * FROM login WHERE adminid = '"+ID+"'";
19            result = state.executeQuery(sql);
20            id1.setText(ID);
21            while (result.next()){
22                name.setText(result.getString(3).toUpperCase());
23                if (result.getBoolean(3)){
24                }
25            }
26        }catch(SQLException e){
27            JOptionPane.showMessageDialog(null, e);
28        }
29    }
30 }

```

MAINPAGE Frame Code

```

178 private void jLabel9MouseClicked(java.awt.event.MouseEvent evt) {
179     int input = JOptionPane.showConfirmDialog(null, "Confirm Logout?",null, JOptionPane.YES_NO_OPTION);
180     if(input == JOptionPane.YES_OPTION) {
181         this.setVisible(false);
182         LOGIN gui = new LOGIN();
183         gui.setTitle("Login");
184         gui.setLocationRelativeTo(null);
185         gui.setVisible(true);
186     }
187 }

```

LOGOUT Code

```

189 private void jLabel1MouseClicked(java.awt.event.MouseEvent evt) {
190     new ADD().setVisible(true);
191 }
192
193 private void jLabel5MouseClicked(java.awt.event.MouseEvent evt) {
194     new RECEIVABLE().setVisible(true);
195 }
196
197 private void jLabel4MouseClicked(java.awt.event.MouseEvent evt) {
198 }
199
200 private void jLabel6MouseClicked(java.awt.event.MouseEvent evt) {
201 }
202
203 private void jLabel2MouseClicked(java.awt.event.MouseEvent evt) {
204     new SEARCH().setVisible(true);
205 }
206
207 private void jLabel3MouseClicked(java.awt.event.MouseEvent evt) {
208     this.setVisible(false);
209     new UPDATE().setVisible(true);
210 }
211

```

JButton Function Call

```

1 package ars;
2 import java.sql.Connection;
3 import java.sql.ResultSet;
4 import java.sql.PreparedStatement;
5 import java.sql.SQLException;
6 import java.text.SimpleDateFormat;
7 import javax.swing.JOptionPane;
8 public class ADD extends javax.swing.JFrame {
9     Connection connect = null;
10    PreparedStatement state = null;
11    ResultSet result = null;
12    String getter;
13    public ADD() {
14        initComponents();
15    }

```

ADD Frame codes

```

203 private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
204     try{
205         connect = ars.DATABASE.connection();
206         String sql = "INSERT INTO `customerinfo`(`name`, `address`, "
207             + "`phone`, `email`, `date`, `amount` ) VALUES (?, ?, ?, ?, ?, ?)";
208         state = connect.prepareStatement(sql);
209         state.setString(1, cname.getText());
210         state.setString(2, caddress.getText());
211         state.setLong(3, Long.parseLong(cphone.getText()));
212         state.setString(4, cemail.getText());
213         SimpleDateFormat sdf = new SimpleDateFormat( "yyyy-MM-dd");
214         String date = sdf.format(pdate.getDate());
215         state.setString(5, date);
216         state.setFloat(6, Float.parseFloat(pamount.getText()));
217         state.executeUpdate();
218         JOptionPane.showMessageDialog(null, "Saved");
219         connect.close();
220
221     }catch ( SQLException e){
222         JOptionPane.showMessageDialog(null, e);
223     }
224 }

```

Insert Code

```

226 private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
227     cname.setText(null);
228     caddress.setText(null);
229     cphone.setText(null);
230     cemail.setText(null);
231     pdate.setDate(null);
232     pamount.setText(null);
233 }
234

```

Clear Textbox Code

```

1 package ars;
2 import java.sql.Connection;
3 import java.sql.DriverManager;
4 import java.sql.PreparedStatement;
5 import java.sql.ResultSet;
6 import java.sql.ResultSetMetaData;
7 import java.sql.SQLException;
8 import java.sql.Statement;
9 import java.util.Vector;
10 import javax.swing.JOptionPane;
11 import javax.swing.table.DefaultTableModel;
12 import javax.swing.table.TableModel;
13 public class UPDATE extends javax.swing.JFrame {
14     Connection connect;
15     Statement state;
16     PreparedStatement pstate;
17     ResultSet result;
18     String getter;
19     public UPDATE() {
20         initComponents();
21     }
22     public Connection Connection(){
23         try{
24             connect = DriverManager.getConnection("jdbc:mysql://localhost/ars","root","");
25             return connect;
26         }
27         catch(SQLException e){
28             return null;
29         }
30     }

```

UPDATE Frame Code


```

33     public final void ShowCustomers()
34     {
35         try{
36             connect = ars.DATABASE.connection();
37             state = connect.createStatement();
38             String sql = "SELECT * FROM customerinfo ";
39             result = state.executeQuery(sql);
40             ResultSetMetaData rsm = result.getMetaData();
41             int c = rsm.getColumnCount();
42             DefaultTableModel df = (DefaultTableModel) this.utable.getModel();
43             df.setRowCount(0);
44             while (result.next()){
45                 Vector rowCell = new Vector();
46                 for (int i = 1; i < c; i++) {
47                     rowCell.add(result.getString(1));
48                     rowCell.add(result.getString(2));
49                     rowCell.add(result.getString(3));
50                     rowCell.add(result.getString(4));
51                     rowCell.add(result.getString(5));
52                     rowCell.add(result.getString(6));
53                     rowCell.add(result.getString(7));
54                 }
55                 df.addRow(rowCell);
56             }
57             result.close();
58         }catch(SQLException e){
59             JOptionPane.showMessageDialog(null, e);
60         }
61     }

```

Show All Data in Table Code

```

320     private void usearchActionPerformed(java.awt.event.ActionEvent evt) {
321         String name = utext.getText().toUpperCase().replaceAll("\\s+", "");
322         try{
323             connect= ars.DATABASE.connection();
324             state = connect.createStatement();
325             String sql = "SELECT * FROM customerinfo WHERE id = '"+name+"' OR name LIKE '%"+name+"%'";
326             result = state.executeQuery(sql);
327             ResultSetMetaData rsm = result.getMetaData();
328             int c = rsm.getColumnCount();
329             DefaultTableModel df = (DefaultTableModel) this.utable.getModel();
330             df.setRowCount(0);
331             while (result.next()){
332                 Vector rowCell = new Vector();
333                 for (int i = 1; i < c; i++) {
334                     rowCell.add(result.getString(1));
335                     rowCell.add(result.getString(2));
336                     rowCell.add(result.getString(3));
337                     rowCell.add(result.getString(4));
338                     rowCell.add(result.getString(5));
339                     rowCell.add(result.getString(6));
340                     rowCell.add(result.getString(7));
341                 }
342                 df.addRow(rowCell);
343             }
344         }catch(SQLException e){
345             JOptionPane.showMessageDialog(null, e);
346         }

```

Search Data Code

```

357     private void cdbuttonActionPerformed(java.awt.event.ActionEvent evt) {
358         int row = utable.getSelectedRow();
359         String a = utable.getModel().getValueAt(row, 0).toString();
360         String sql = "DELETE FROM customerinfo WHERE id = " + a;
361         try{
362             pstate = connect.prepareStatement(sql);
363             pstate.execute();
364             JOptionPane.showMessageDialog(null, "Deleted Successfully");
365         }catch (Exception e){
366             JOptionPane.showMessageDialog(null, e);
367         }finally{
368             try{
369                 pstate.close();
370                 result.close();
371             }catch(Exception e){
372             }
373         }
374         ShowCustomers();
375         cname.setText(null);
376         caddress.setText(null);
377         cphone.setText(null);
378         cemail.setText(null);
379         cdate.setText(null);
380         camount.setText(null);
381         cid.setText(null);
382     }

```

Delete Code

```

388 private void cupbuttonMouseClicked(java.awt.event.MouseEvent evt) {
389     try{
390         connect = ars.DATABASE.connection();
391         String sql = "UPDATE customerinfo SET name= ?,address = ?,phone = ?,\"
392             + \"email = ?,date = ?, amount = ? WHERE id = ?\" ;
393         pstate = connect.prepareStatement(sql);
394         pstate.setString(7, cid.getText());
395         pstate.setString(1, cname.getText());
396         pstate.setString(2, caddress.getText());
397         pstate.setString(3, cphone.getText());
398         pstate.setString(4, cemail.getText());
399         pstate.setString(5, cdate.getText());
400         pstate.setString(6, camount.getText());
401         pstate.executeUpdate();
402         JOptionPane.showMessageDialog(null, "Updated Succesfully");
403         ShowCustomers();
404     }
405     catch (SQLException ex) {
406         JOptionPane.showMessageDialog(null, ex);
407     }
408     cname.setText(null);
409     caddress.setText(null);
410     cphone.setText(null);
411     cemail.setText(null);
412     cdate.setText(null);
413     camount.setText(null);
414     cid.setText(null);
415 }
416

```

Update Data Code

```

418 private void utableMouseClicked(java.awt.event.MouseEvent evt) {
419     utable.setDefaultEditor(Object.class, null);
420     TableModel model = (TableModel)utable.getModel();
421     int a = utable.getSelectedRow();
422     cid.setText(model.getValueAt(a, 0).toString());
423     cname.setText(model.getValueAt(a, 1).toString());
424     caddress.setText(model.getValueAt(a, 2).toString());
425     cphone.setText(model.getValueAt(a, 3).toString());
426     cemail.setText(model.getValueAt(a, 4).toString());
427     cdate.setText(model.getValueAt(a, 5).toString());
428     camount.setText(model.getValueAt(a, 6).toString());
429 }

```

JTable Code

```

1 package ars;
2 import java.sql.Connection;
3 import java.sql.PreparedStatement;
4 import java.sql.ResultSet;
5 import java.sql.ResultSetMetaData;
6 import java.sql.SQLException;
7 import java.sql.Statement;
8 import java.util.Vector;
9 import javax.swing.JOptionPane;
10 import javax.swing.table.DefaultTableModel;
11 import net.proteanit.sql.DbUtils;
12 public class RECEIVABLE extends javax.swing.JFrame {
13     Connection connect = null;
14     Statement state = null;
15     PreparedStatement pstate = null;
16     ResultSet result = null;
17     String getter;
18     public RECEIVABLE() {
19         initComponents();
20     }

```

RECEIVABLE Frame Code

```

33 public void TotalAmount(){
34     float sum = 0;
35     for(float i =0; i<rtable.getRowCount(); i++){
36         sum = sum + Float.parseFloat(rtable.getValueAt((int) i, 6).toString());
37     }
38     total.setText(Float.toString((float) sum));
39 }

```

Calculation of Total Amount Code

```

1 package ars;
2 import java.sql.Connection;
3 import java.sql.PreparedStatement;
4 import java.sql.ResultSet;
5 import java.sql.ResultSetMetaData;
6 import java.sql.SQLException;
7 import java.sql.Statement;
8 import java.util.Vector;
9 import javax.swing.JOptionPane;
10 import javax.swing.table.DefaultTableModel;
11 import net.proteanit.sql.DbUtils;
12 public class SEARCH extends javax.swing.JFrame {
13     Connection connect = null;
14     Statement state = null;
15     PreparedStatement pstate = null;
16     ResultSet result = null;
17     String getter;
18     public SEARCH() {
19         initComponents();
20     }
21     public final void ShowCustomers()
22     {
23         try{
24             connect = DATABASE.connection();
25             String sql = "SELECT * FROM customerinfo";
26             pstate = connect.prepareStatement(sql);
27             result = pstate.executeQuery();
28             stable.setModel(DbUtils.resultSetToTableModel(result));
29         }catch(SQLException ex){
30             JOptionPane.showMessageDialog(null, ex);
31         }
32     }

```

SEARCH Frame Code

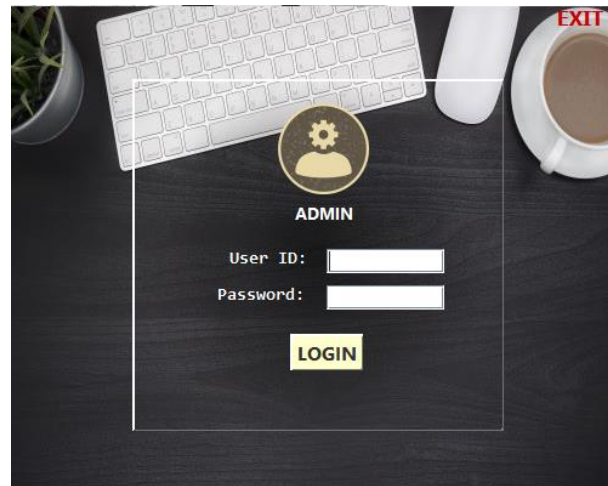
```

198 private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
199     String name = search.getText().toUpperCase().replaceAll("\\s+", "");
200     try{
201         connect= ars.DATABASE.connection();
202         state = connect.createStatement();
203         String sql = "SELECT * FROM customerinfo WHERE id = '"+name+"' OR name LIKE '%"+name+"%'";
204         result = state.executeQuery(sql);
205         ResultSetMetaData rsm = result.getMetaData();
206         int c = rsm.getColumnCount();
207         DefaultTableModel df = (DefaultTableModel) this.stable.getModel();
208         df.setRowCount(0);
209         while (result.next()){
210             Vector rowCell = new Vector();
211             for (int i = 1; i < c; i++) {
212                 rowCell.add(result.getString(1));
213                 rowCell.add(result.getString(2));
214                 rowCell.add(result.getString(3));
215                 rowCell.add(result.getString(4));
216                 rowCell.add(result.getString(5));
217                 rowCell.add(result.getString(6));
218                 rowCell.add(result.getString(7));
219             }
220             df.addRow(rowCell);
221         }
222     }catch(SQLException e){
223         JOptionPane.showMessageDialog(null, e);
224     }
225 }

```

Search Code

SCREEN OUTPUT



ADMIN

User ID:

Password:

LOGIN

EXIT

STEP 1



ACCOUNTING RECEIVABLE

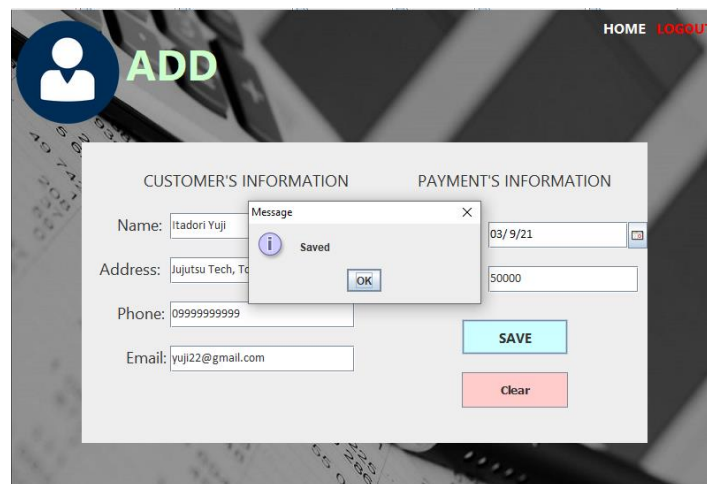
Admin: JIREH CABRESTANTE
ID: 111111

ADD **UPDATE**

RECEIVABLE **SEARCH**

LOGOUT

STEP 2



ADD HOME LOGOUT

CUSTOMER'S INFORMATION

Name:

Address:

Phone:

Email:

PAYMENT'S INFORMATION

Date:

Amount:

SAVE **Clear**

Message Saved OK

STEP 3



UPDATE

[HOME](#)
[LOGOUT](#)

(Search by ID or NAME)

ID	NAME	ADDRESS	PHONE	EMAIL	DATE	AMOUNT
28	zzzzzz	xxxxxx	633588	aa	2021-02-24	100.25
29	wwwww	xxxxxx	633588	aa	2021-02-24	52000
31	aa	aaaaaa	6392598754	iguyfk56154@fff	2021-02-25	800.5
32	aaaaaaa	bbbbbbbbbbb...	6392598754	ddd@amai.com	2020-02-24	100
34	aaaaa	aaaaa212322....	6392558	erf@aamilcmo	2021-02-24	1
35	Itadori Yuui	Juiutsu Tech. T...	9999999999	vuui22@amail.c...	2021-03-09	50000

ID:

Email:

Name:

Date:

Address:

Amount:

Phone:

STEP 4



UPDATE

[HOME](#)
[LOGOUT](#)

(Search by ID or NAME)

ID	NAME	ADDRESS	PHONE	EMAIL	DATE	AMOUNT
31	aa	aaaaaa	6392598754	iguyfk56154@fff	2021-02-25	800.5

ID: 31

Email: iguyfk56154@fff

Name: aa

Date: 2021-02-25

Address: aaaaaa

Amount: 800.5

Phone: 6392598754

STEP 5



UPDATE

[HOME](#)
[LOGOUT](#)

(Search by ID or NAME)

ID	NAME	ADDRESS	PHONE	EMAIL	DATE	AMOUNT
31	aa	bbbb	99999999	iguyfk56154@fff	2021-02-25	800.525

ID: 31

Email: iguyfk56154@fff

Name: aa

Date: 2021-02-25

Address: bbbb

Amount: 800.525

Phone: 99999999

Message


Updated Successfully

STEP 6



UPDATE

HOME [LOGOUT](#)

(Search by ID or NAME)

ID	NAME	ADDRESS	PHONE	EMAIL	DATE	AMOUNT
28	zzzzzz	xxxxxx	633588	aa	2021-02-24	100.25
29	wwwwww	xxxxxx	633588	aa	2021-02-24	52000
31	aa	bbbb	99999999	iguyfk56154@fff	2021-02-25	800.525
32	aaaaaaa	bbbbbbbbbbbbb...	6392598754	ddd@gmail.com	2020-02-24	100
34	aaaaa	aaaaa212322....	6392558	erf@gmailc...	2021-02-24	1
35	Itadori Yuui	Jujutsu Tech. T...	9999999999	yuui22@gmail.c...	2021-03-09	50000

ID: 31

Email:

Name:

Date:

Address:

Amount:

Phone:

STEP 7



UPDATE

HOME [LOGOUT](#)

(Search by ID or NAME)

ID	NAME	ADDRESS	PHONE	EMAIL	DATE	AMOUNT
28	zzzzzz	xxxxxx	633588	aa	2021-02-24	100.25
29	wwwwww	xxxxxx	633588	aa	2021-02-24	52000
31	aa	bbbb	99999999	iguyfk56154@fff	2021-02-25	800.525
32	aaaaaaa	bbbbbbbbbbbbb...	6392598754	ddd@gmail.com	2020-02-24	100
34	aaaaa	aaaaa212322....	6392558	erf@gmailc...	2021-02-24	1
35	Itadori Yuui	Jujutsu Tech. T...	9999999999	yuui22@gmail.c...	2021-03-09	50000

ID: 31

Email:

Name:

Date:

Address:

Amount:

Phone:

STEP 8



RECEIVABLE

HOME [LOGOUT](#)

Search by YEAR (2020) or MONTH (2020-11)

id	name	address	phone	email	date	amount
28	zzzzzz	xxxxxx	633588	aa	2021-02-24	100.25
29	wwwwww	xxxxxx	633588	aa	2021-02-24	52000.0
32	qqqqqqq	bbbbbbbbbbbbb...	6392598754	ddd@gmail.com	2020-02-24	100.0
34	qqqqq	qqqqq212322....	6392558	erf@gmailc...	2021-02-24	1.0
35	Itadori Yuji	Jujutsu Tech. T...	9999999999	yuji22@gmail.c...	2021-03-09	50000.0

STEP 9



RECEIVABLE

[HOME](#)
[LOGOUT](#)

Search by YEAR (2020) or MONTH (2020-11)

ID	NAME	ADDRESS	PHONE	EMAIL	DATE	AMOUNT
28	zzzzzz	xxxxx	633588	aa	2021-02-24	100.25
29	www	xxxxx	633588	aa	2021-02-24	52000
34	qqqqq	qqqqq212322...	6392558	erf@qamilc...	2021-02-24	1
35	Itadori Yuji	Jujutsu Tech, T...	9999999999	yuji22@gmail.c...	2021-03-09	50000

STEP 10



RECEIVABLE

[HOME](#)
[LOGOUT](#)

Search by YEAR (2020) or MONTH (2020-11)

ID	NAME	ADDRESS	PHONE	EMAIL	DATE	AMOUNT
28	zzzzzz	xxxxx	633588	aa	2021-02-24	100.25
29	www	xxxxx	633588	aa	2021-02-24	52000
34	qqqqq	qqqqq212322...	6392558	erf@qamilc...	2021-02-24	1

STEP 11



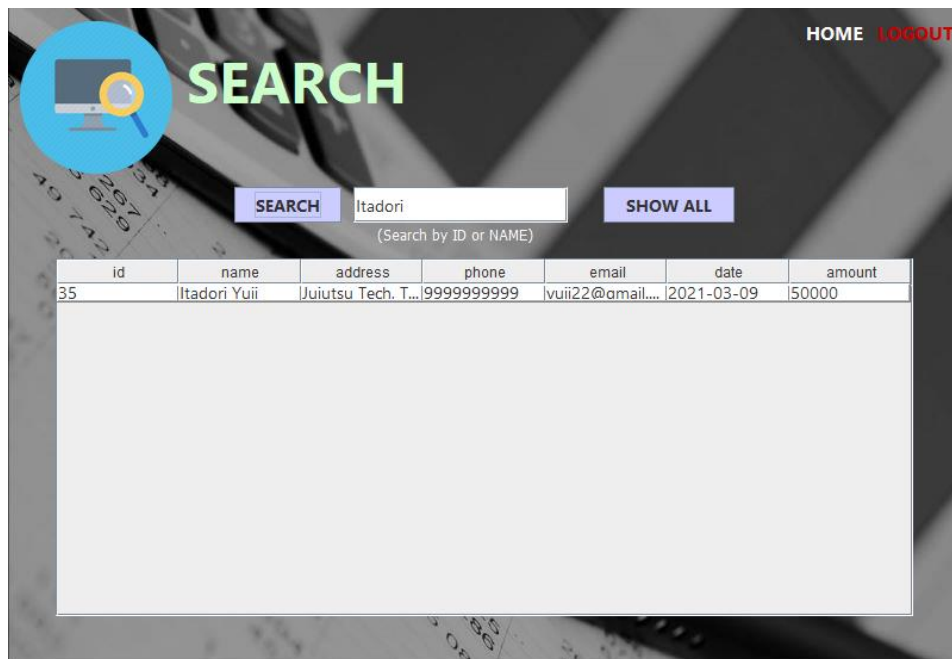
SEARCH

[HOME](#)
[LOGOUT](#)

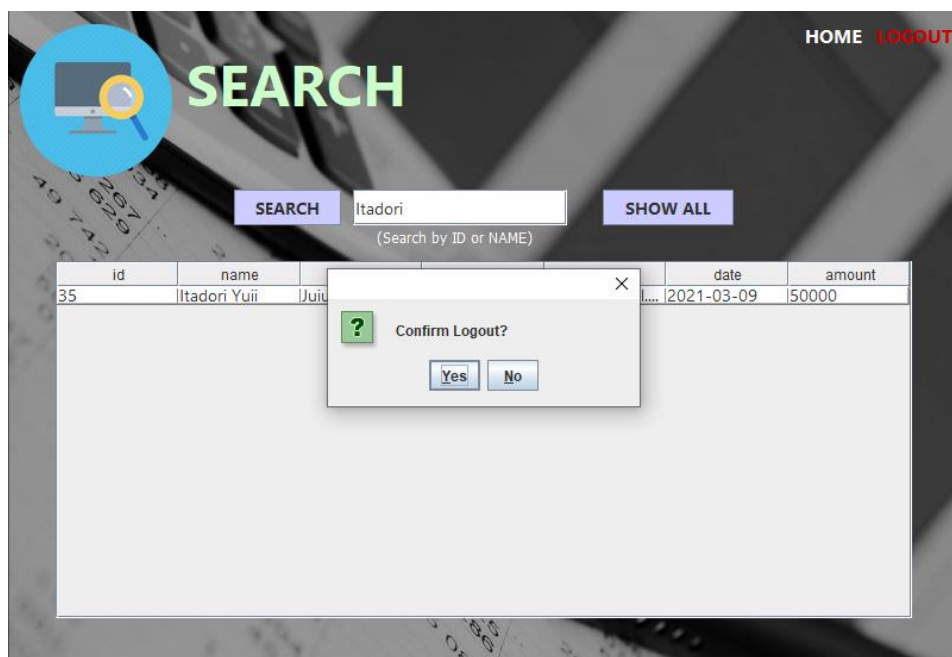
(Search by ID or NAME)

id	name	address	phone	email	date	amount
28	zzzzzz	xxxxx	633588	aa	2021-02-24	100.25
29	www	xxxxx	633588	aa	2021-02-24	52000.0
32	aaaaaa	bbbbbbbbbb...	6392598754	ddd@qmai.com	2020-02-24	100.0
34	aaaaa	aaaaa212322...	6392558	erf@qamilc...	2021-02-24	1.0
35	Itadori Yuji	Juiutsu Tech, T...	9999999999	vuji22@mail...	2021-03-09	50000.0

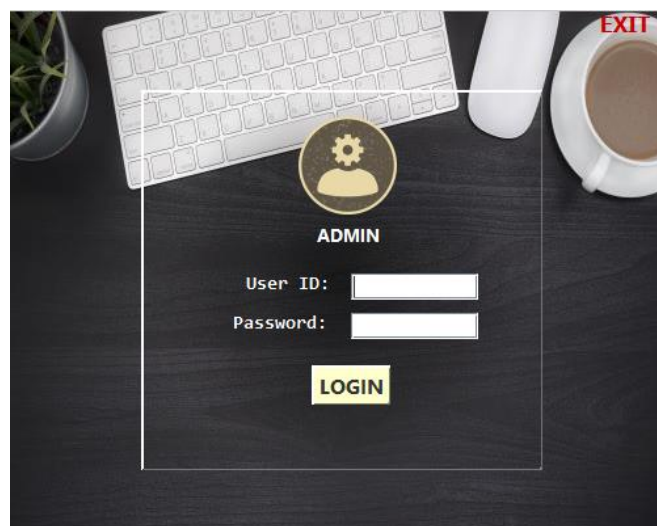
STEP 12



STEP 13



STEP 14



STEP 15

REFERENCES

https://www.researchgate.net/publication/322942500_Development_of_Account_Receivable_and_Payable_System_for_Travel_Bureau_Company

<https://sba.thehartford.com/finance/accounting/accounts-receivable-process/>

<https://ofm.wa.gov/it-systems/accounts-receivable-system-ar>